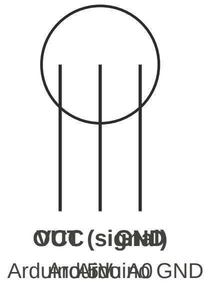
CanSat — Pin-by-Pin Wiring Reference

Every component, every physical pin, and exactly where it goes on the Arduino

1. LM35 — Temperature Sensor (3 legs)

front view (flat side facing you)



Component pin	Goes to on the Arduino	Note
Leg 1 (left) = VCC	Arduino 5V	regulated bus, shared with the other sensors
Leg 2 (middle) = OUT	Arduino A0	analog temperature signal
Leg 3 (right) = GND	Arduino GND	common ground

Warning: this part has no printed label. If wired backwards it can overheat — double-check orientation before powering it.

2. BMP280 — Altitude Sensor (barometer)

Component pin	Goes to on the Arduino	Note
VCC	Arduino 5V	regulated bus
GND	Arduino GND	common ground
SCL	Arduino A5	I2C clock line
SDA	Arduino A4	I2C data line — shared with the GY-521
CSB (if present)	Arduino 5V	ties high — keeps the sensor in I2C mode
SDO (if present)	Arduino GND	sets the I2C address to 0x76, matching the example c

Do not leave CSB/SDO floating — an unconnected address-select pin can read an unpredictable logic level and cause intermittent I2C erro

3. GY-521 / MPU6050 — Acceleration Sensor

Component pin	Goes to on the Arduino	Note
VCC	Arduino 5V	regulated bus
GND	Arduino GND	common ground
SCL	Arduino A5	same line as the BMP280 (parallel/I2C)
SDA	Arduino A4	same line as the BMP280 (parallel/I2C)
AD0	Arduino GND	sets the I2C address to 0x68, matching the example c
XDA, XCL, INT	do not use	leave disconnected

Do not leave AD0 floating — same reason as BMP280's SDO: an undefined address-select pin can cause intermittent I2C errors.

The BMP280's and GY-521's SCL and SDA go to the EXACT SAME A4/A5 pins on the Arduino — both sensors share the same wire (para

4. Servo SG90 — Parachute Release (3 wires)

Wire (color)	Goes to on the Arduino	Note
Brown/black wire	Arduino GND	common ground
Red wire	Arduino 5V	regulated bus
Orange/yellow wire	Arduino D9	PWM angle control signal

5. XBee Explorer — Telemetry Radio

Component pin	Goes to on the Arduino	Note
5V	Arduino 5V — VERIFY FIRST (see note below)	regulated bus
GND	Arduino GND	common ground
OUT	Arduino D0 (RX)	data leaving the XBee to the Arduino
IN	Arduino D1 (TX)	data leaving the Arduino to the XBee
DIO0–DIO12, RTS, CTS, RES, RSSI, DTR, RST, 3.3V	do not use	leave all disconnected

Before uploading new code, disconnect the D0 and D1 wires — those pins are also used by the USB cable.

IMPORTANT — check your exact board first: some 'XBee Explorer' breakouts (unlike the 'Explorer Regulated' version) do NOT step 5V down to 3.3V internally. The XBee module itself needs 3.3V — feeding it 5V on an unregulated board can damage it. If you are not certain your board regulates the voltage, power it from the Arduino's 3V3 pin instead of 5V.

6. L298N Motor Driver

Component pin	Goes to on the Arduino	Note
IN1	Arduino D2	Motor A direction control
IN2	Arduino D3	Motor A direction control
IN3	Arduino D4	Motor B direction control
IN4	Arduino D5	Motor B direction control
ENA	5V jumper (default) or Arduino PWM pin	enables Motor A — leave the onboard jump
ENB	5V jumper (default) or Arduino PWM pin	enables Motor B — leave the onboard jump
12V / VMS (+ terminal)	Battery 9V (+) direct	motor power — does NOT go through the A
GND terminal	Battery 9V (–) / common GND	same ground as everything else
5V output pin	do not use to power sensors	only active if the onboard regulator jumper i
OUT1 / OUT2	Motor A wires	power output
OUT3 / OUT4	Motor B wires	power output

Most L298N boards ship with the ENA/ENB jumpers already installed, which enables both motors at full speed by default. Only remove them if you want variable speed control by connecting ENA/ENB to a PWM-capable Arduino pin instead.

7. 9V Battery

Terminal	Goes to	Note
Positive terminal (red)	Arduino VIN + Motor driver (+)	powers both at once, in parallel
Negative terminal (black)	Arduino GND + Motor driver (–)	same common ground for the whole ci

8. ESP32-CAM (Camera)

8.1 Power

Component pin	Goes to on the Arduino	Note
5V	Battery — direct (raw) line	do NOT use the same regulated bus as the ser
GND	common GND	same ground as everything

Why the power is separate: the camera + Wi-Fi draw current spikes similar to the motor. If it shared the sensors' regulated bus, it could sag the voltage while taking a photo and disrupt the BMP280/GY-521 readings.

8.2 Communication with the Arduino (data)

Component pin	Goes to on the Arduino	Note
U0T (TX)	Arduino D7 (SoftwareSerial RX)	ESP32-CAM sends data to the Arduino
U0R (RX)	Arduino D6 (SoftwareSerial TX)	Arduino sends commands to the ESP32-CAM

These pins were moved off D0/D1 on purpose: the XBee already uses the Arduino's hardware serial (D0/D1). Using D6/D7 with the Arduino SoftwareSerial library lets the camera and the XBee communicate at the same time without conflict.

8.3 Flashing the code (pre-flight only)

Component pin	Goes to on the Arduino	Note
IO0	GND (only during upload)	enters flashing mode
5V / GND / U0R / U0T	FTDI USB-Serial adapter	required to flash — not included in your kit

After flashing, DISCONNECT IO0 from GND before powering up for flight — if it stays grounded, the board gets stuck in flashing mode and won't run the program.

8.4 Pins that should NOT be used

Component pin	Goes to on the Arduino	Note
IO12, IO13, IO14, IO15	do not connect	used internally by the camera
IO2, IO4, IO16	do not connect	used internally by the SD card

Before finalizing this in solder, decide as a team:

1) Are you implementing the camera in this phase, or leaving it for a future iteration?

2) Do you have the FTDI adapter? Without it, the ESP32-CAM cannot be flashed.